A Score Ceiling Diagnoses a Dead Retriever in Reciprocal Rank Fusion

Benjamin Meadows
Independent Researcher
ben@benjaminmeadows.com

October 2026

Abstract

A hybrid search system fuses a keyword retriever and a vector retriever with reciprocal rank fusion (RRF). In the system described here, the keyword retriever returned no results for most natural-language queries for several months, and nobody noticed: the vector retriever always filled the result list, and the usual spot checks used short keyword queries, which still matched. The failure was visible the whole time in the fused scores. RRF credits a document $1/(k + r)$ from each retriever that ranks it at position r, so a document found by only one retriever scores at most $1/k$. When the maximum fused score across a result list is exactly $1/k$, no document was found by more than one retriever; across a set of queries that ceiling is the signature of a retriever that returns nothing, and it can be read from the scores alone. I give the derivation, the bug that caused it (quoting every token of an SQLite FTS5 query, which FTS5 reads as an implicit AND), before-and-after measurements on the production corpus, and two negative results from the same system that the dead leg had confounded. I then reproduce the effect on two public BEIR collections with two encoders: the quoted-AND builder returns nothing for 98.0% of SciFact's sentence queries and 50.8% of NFCorpus's title queries, the hybrid built on it is indistinguishable from dense-only retrieval on the sentence-query collection, and the fraction of queries whose maximum fused score sits at $1/k$ tracks the fraction with an empty keyword leg to within one percentage point. Code, results and this paper's source are at [repository URL].

1 The incident

The system is a personal search engine over about 30 000 to 217 000 text chunks per corpus, split across five SQLite databases that share one retrieval module. Each query runs two retrievers over the same chunks: an SQLite FTS5 index ranked by BM25 [Robertson and Zaragoza, 2009, SQLite Consortium], and a 768-dimensional embedding index searched by exact cosine. The two ranked lists, each cut at depth $K = 60$, are fused with reciprocal rank fusion at $k = 60$ [Cormack et al., 2009].

On 2026-08-07 a recency-decay feature was evaluated and rejected because RRF scores were "flat near the top": adjacent ranks differed by about 1.4%, so even a 5% age multiplier acted as a strong reranker rather than a tie-break. That flatness was recorded as a property of fusion. On 2026-08-18, while investigating why the keyword side rarely seemed to matter, I logged the keyword retriever's row counts for ten real queries. Five of five conversational queries matched zero chunks, as did a two-word place-name query. Only queries of two or three technical tokens matched

anything. The keyword retriever had been inert on prose queries since the system was built, and all four sibling engines carried the same line of code.

The fused scores had said so from the start. Over the top 100 fused results of those ten queries, the maximum score was exactly $1/60 = 0.01\overline{6}$. The values were $1/60, 1/61, 1/62, \ldots$ with gaps where per-file deduplication removed a chunk. That is one live retriever's rank list passed through the RRF formula, and the "1.4% flatness" is its relative step, $\big(1/(k+r) - 1/(k+r+1)\big)/\big(1/(k+r)\big) = 1/(k+r+1)$, which is 1.6% at the top of the list and 1.4% around rank 10.

2 The ceiling

Let L retrievers each return a ranked list cut at depth K, and let $r_\ell(d) \in \{0, \ldots, K-1\}$ be document d's 0-based rank in retriever ℓ's list when present. Reciprocal rank fusion scores

$$s(d) \;=\; \sum_{\ell : d \in \text{list}_\ell} \frac{1}{k + r_\ell(d)}.$$

A document present in exactly one list scores at most $1/k$ (rank 0). A document present in two lists scores at least $2/(k+K-1)$ (last place in both). The second bound exceeds the first whenever

$$\frac{2}{k+K-1} > \frac{1}{k} \iff K \le k,$$

so with the common setting $K \le k$ (here $K = k = 60$), every document found by two or more retrievers outscores every document found by one. Under that setting the following holds exactly:

> If $\max_d s(d) \le 1/k$ over a result list, no document in it was retrieved by more than one retriever.

For one query that is weak evidence: the retrievers may simply disagree. Across a set of queries it is strong evidence, because two retrievers over the same corpus that both return K results almost always share at least one document somewhere in their top K. A retriever that returns nothing, however, shares nothing with anyone, every time. In the incident, the ceiling held on 10 of 10 queries, and the keyword leg had returned zero rows on 6 of them and fewer than K on the rest.

When $K > k$ the exact statement fails: a document ranked deep in both lists can score below $1/k$, and one ranked exactly k in both scores exactly $1/k$ (for $k = 60$, $1/120 + 1/120$). Then a maximum of $1/k$ no longer proves a single-leg list, and the test is read in its weak form: the sequence $1/k, 1/(k+1), 1/(k+2), \ldots$ running down the fused scores is still the fingerprint of one list, and a system that keeps per-retriever ranks tests the condition directly by counting documents credited by two or more retrievers. Section 6 uses $K = 100 > k$ and does both. The point of the ceiling is that a system that logs only fused scores can run the exact test for $K \le k$, and the fingerprint test otherwise, at no cost.

The relation between the two rates reported later also needs one line. The ceiling fires whenever the keyword leg is empty, provided the dense leg returned at least one document (its rank-0 document then scores exactly $1/k$ and nothing can exceed it), so the set of empty-leg queries is a subset of the at-ceiling queries, and the difference between the two rates counts queries on which the keyword leg returned documents that shared nothing with the dense top K: the test's false positives.

3 The bug

The keyword query builder was

```
fq = " ".join(f'"{t}"' for t in tokens)
```

which, for the query *how do I fix the empty feed*, produced

```
"how" "do" "i" "fix" "the" "empty" "feed"
```

FTS5 reads a sequence of phrases separated by whitespace as an implicit AND [SQLite Consortium, §3]: every quoted token must occur in the same row. Rows here were 1 400-character chunks, so a six-token conversational query almost never matched. Two- or three-token technical queries did match, which is why spot checks during development passed, and an FTS5 syntax error was caught and ignored, so a malformed query also returned nothing without a trace. The vector retriever returns exactly K results for any query, so result lists were always full.

The fix has two parts, and the second is not cosmetic. Tokens are joined with `OR` and ranked by BM25; a raw OR over all tokens matched about 100 000 chunks and cost 161 ms per query, so stopwords and tokens shorter than three characters are dropped first, after which a typical query matches about 3 500 chunks in 2 to 4 ms. If a query is all stopwords, the raw tokens are used.

4 Measurements on the production corpus

The corpus is private; this section reports counts, timings and overlaps, with no query or document text. Ten real queries were replayed before and after the fix on the same shard (Table 1).

	implicit AND	OR + stopword drop
Queries on which the keyword leg returned any row	4 / 10	10 / 10
Queries with a document found by both legs	0 / 10	10 / 10
Maximum fused score over the top 100 results	$1/60 = 0.0167$	0.025 – 0.033
Keyword leg latency, conversational queries	–	2 – 4 ms

Table 1: The keyword leg before and after the fix, ten real queries, one production shard. The "before" latency is omitted because a query that matches nothing is fast; a raw OR without the stopword drop took 161 ms.

The same builder was present at a different line number in four sibling engines over four other corpora, each inert on prose queries in the same way; the five have since been merged into one module.

5 Two negative results the dead leg confounded

Both of the following were measured on the same system before the keyword leg was fixed, and both verdicts stand; what changed is the explanation.

Recency decay. A multiplicative age penalty on fused scores (floor 0.85, time constant 180 days) changed the top-1 result on 4 of 7 real queries on a notes corpus, and the changes were as often wrong as right; a canonical note unedited for 338 days was demoted under a 13-day-old draft. On a second corpus spanning only 63 days the multiplier varied between 0.956 and 1.000 across every document and changed 0 of 8 top-1 results. The feature was rejected. The reason a 5% multiplier could move top-1 at all was the flatness of the fused scores, and that flatness was the dead leg: a single retriever's $1/(k+r)$ sequence. Whether decay would help a fused list with real two-leg scores was not tested and is not claimed either way.

A cheap contextual prefix. Prepending `kind: title` to each chunk before embedding (the stored text and the keyword index unchanged, so the vector was the only variable) and re-embedding one corpus of 3 046 documents and 30 734 chunks took 25 minutes. On 15 realistic queries, 13 had an identical top-1, mean top-5 overlap was 80%, one query improved and one regressed. The query predicted to benefit most returned an identical, still wrong, top 3. The change was not adopted. This is a title prefix, not the model-written situating context of Anthropic [2024], and the corpus was a weak test (documents whose title is their opening line), so it is a negative result for the cheap variant on that corpus, nothing wider. On this system the vector leg was the only live leg at the time, so the measurement is of the vector change alone, which was the intent.

6 Reproduction on public collections

To check that the effect is a property of the query builder and not of one private corpus, I reran the five retrieval conditions on two BEIR test collections [Thakur et al., 2021]: SciFact [Wadden et al., 2020], 5 183 abstracts and 300 claim queries, and NFCorpus [Boteva et al., 2016], 3 633 documents and 323 queries. SciFact queries are full sentences; NFCorpus queries are short titles. Two encoders were used: `nomic-embed-text-v1.5` [Nussbaum et al., 2024], the production encoder's public release, and `bge-base-en-v1.5` [Xiao et al., 2023], both 768-dimensional. Data came through `ir_datasets` [MacAvaney et al., 2021] and metrics through `pytrec_eval` [Van Gysel and de Rijke, 2018]. Depth $K = 100$, $k = 60$; note that $K > k$ here, so the ceiling is read in its weak form and the two-leg count is measured directly.

The conditions are: BM25-AND (the buggy builder), BM25-OR (the fix), DENSE, HYBRID-AND (dense + buggy keyword, RRF), HYBRID-OR (dense + fixed keyword, RRF). For each hybrid condition the per-query diagnostics are whether the keyword leg returned nothing, whether the maximum fused score sat at $1/k$, and how many documents were credited by both legs.

Three things are visible in Table 2. First, the builder's failure depends on query length exactly as it did in production: SciFact's claim-length queries matched nothing 98.0% of the time, NFCorpus's short title queries 50.8% of the time, and BM25 with the fixed builder is a respectable retriever on both (0.665 and 0.306 nDCG@10). Second, where the leg is empty the hybrid is the dense retriever under another name: on SciFact, HYBRID-AND has identical Recall@100 to DENSE and nDCG@10 within 0.002, because a fusion with an empty list is the other list. On NFCorpus the leg fired on half the queries, and HYBRID-AND differs from DENSE by under 0.005 Recall@100 and under 0.008 nDCG@10, the contribution of that half. Third, the ceiling detects the empty leg almost exactly. On SciFact the two rates coincide (98.0% and 98.0%); on NFCorpus the ceiling fires on 51.7% of queries against 50.8% empty, three queries on which the keyword leg returned documents that shared nothing with the dense top 100, and after the fix 8.4% against 7.7%, two queries. Those are the false positives of the test on these collections: under one percentage point.

Collection	Condition	nDCG@10	R@100	kw empty	at ceiling	both-leg docs
SciFact	bm25-and	0.018	0.018	98.0%	–	–
SciFact	bm25-or	0.665	0.876	0.0%	–	–
SciFact	dense	0.738	0.970	–	–	–
SciFact	hybrid-and	0.739	0.970	98.0%	98.0%	0.02
SciFact	hybrid-or	0.730	0.969	0.0%	0.0%	28.4
NFCorpus	bm25-and	0.193	0.095	50.8%	–	–
NFCorpus	bm25-or	0.306	0.233	7.7%	–	–
NFCorpus	dense	0.369	0.332	–	–	–
NFCorpus	hybrid-and	0.377	0.337	50.8%	51.7%	3.9
NFCorpus	hybrid-or	0.358	0.336	7.7%	8.4%	20.2

Table 2: `bge-base-en-v1.5` on two BEIR test collections. "kw empty" is the fraction of queries on which the keyword leg returned no document; "at ceiling" the fraction on which the maximum fused score was $1/k$; "both-leg docs" the mean number of documents credited by both retrievers. The second encoder is in Appendix A.

The fix restores the keyword leg; it does not, on these two collections with this encoder, make the hybrid better than dense alone at nDCG@10 (0.730 against 0.738 on SciFact, 0.358 against 0.369 on NFCorpus). On NFCorpus the half-dead leg did better than the live one at nDCG@10 (0.377 against 0.358): the implicit-AND builder fired only on short, specific queries, where BM25 is at its best, and stayed silent where it would have added noise. With the other encoder (Appendix A) the hybrid gains three points of Recall@100 on SciFact and is otherwise within noise; the weaker dense encoder is the one the keyword leg helps. Whether RRF over BM25 and a given dense encoder helps is a question Bruch et al. [2023] treat at length, and with two encoders and two collections this paper cannot settle it. The point here is narrower. Before the fix, the question could not be asked, and the system's own scores had been saying so.

7 Related work

Reciprocal rank fusion was proposed by Cormack et al. [2009], who set $k = 60$ by experiment; that constant has survived into most hybrid-search implementations unchanged, which is what makes the 1/60 ceiling recognisable on sight. The older fusion literature studied overlap between the lists being fused: CombSUM and CombMNZ [Fox and Shaw, 1994] reward documents returned by several systems, and Lee [1997] showed that fusion helps when the systems agree on relevant documents more than on non-relevant ones. The both-leg count in Section 6 is that overlap, measured where it had silently gone to zero. Bruch et al. [2023] analyse fusion functions for hybrid lexical and semantic retrieval and show when RRF is and is not competitive with a tuned convex combination; this paper is about a failure mode upstream of that choice, where one of the lists is empty. Kamphuis et al. [2020] show how much BM25 scores vary across implementations of the same formula; the FTS5 variant here is one more, and its query grammar, not its scoring, is what failed. BEIR [Thakur et al., 2021] established that BM25 is a hard zero-shot baseline, which is part of why a hybrid system that silently drops it loses more than it appears to. The FTS5 query grammar is documented in the SQLite manual [SQLite Consortium]; the implicit-AND reading of adjacent phrases is stated there, and the builder here simply did not follow it.

8 Limitations

The production measurements are on ten and fifteen queries over private corpora, reported as aggregates; they establish that the failure happened and how it looked, not an effect size. The public reproduction uses two small collections and two encoders, with FTS5's default BM25 weighting of title and text. The recency and prefix results are negative results on one system each and are not claims about those techniques in general.

9 What to check in your own system

Log the maximum fused score per query. If it is $1/k$ more than occasionally, one of your retrievers is returning nothing. Log each retriever's row count per query alongside it; an FTS layer that returns zero rows for a six-word query is broken, not selective. Test the keyword builder with a conversational sentence, not a two-word term. And if your fusion scores look "flat", compute $1/(k+r) - 1/(k+r+1)$ before concluding anything about fusion.

References

Anthropic. Introducing contextual retrieval. `https://www.anthropic.com/news/contextual-retrieval`, 2024. Accessed 2026-10-07.

Vera Boteva, Demian Gholipour, Artem Sokolov, and Stefan Riezler. A full-text learning to rank dataset for medical information retrieval. In *Advances in Information Retrieval: 38th European Conference on IR Research*, pages 716–722, 2016.

Sebastian Bruch, Siyu Gai, and Amir Ingber. An analysis of fusion functions for hybrid retrieval. *ACM Transactions on Information Systems*, 42(1), 2023.

Gordon V. Cormack, Charles L. A. Clarke, and Stefan Büttcher. Reciprocal rank fusion outperforms Condorcet and individual rank learning methods. In *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 758–759, 2009.

Edward A. Fox and Joseph A. Shaw. Combination of multiple searches. In *Proceedings of the Second Text REtrieval Conference (TREC-2)*, pages 243–252, 1994.

Chris Kamphuis, Arjen P. de Vries, Leonid Boytsov, and Jimmy Lin. Which BM25 do you mean? a large-scale reproducibility study of scoring variants. In *Advances in Information Retrieval: 42nd European Conference on IR Research*, pages 28–34, 2020.

Joon Ho Lee. Analyses of multiple evidence combination. In *Proceedings of the 20th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 267–276, 1997.

Sean MacAvaney, Andrew Yates, Sergey Feldman, Doug Downey, Arman Cohan, and Nazli Goharian. Simplified data wrangling with ir_datasets. In *Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 2429–2436, 2021.

Zach Nussbaum, John X. Morris, Brandon Duderstadt, and Andriy Mulyar. Nomic embed: Training a reproducible long context text embedder. *arXiv preprint arXiv:2402.01613*, 2024.

Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009.

SQLite Consortium. SQLite FTS5 extension. `https://www.sqlite.org/fts5.html`. Section 3, full-text query syntax. Accessed 2026-10-07.

Nandan Thakur, Nils Reimers, Andreas Rücklé, Abhishek Srivastava, and Iryna Gurevych. BEIR: A heterogeneous benchmark for zero-shot evaluation of information retrieval models. In *Thirty-fifth Conference on Neural Information Processing Systems, Datasets and Benchmarks Track*, 2021.

Christophe Van Gysel and Maarten de Rijke. Pytrec_eval: An extremely fast python interface to trec_eval. In *Proceedings of the 41st International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 873–876, 2018.

David Wadden, Shanchuan Lin, Kyle Lo, Lucy Lu Wang, Madeleine van Zuylen, Arman Cohan, and Hannaneh Hajishirzi. Fact or fiction: Verifying scientific claims. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*, pages 7534–7550, 2020.

Shitao Xiao, Zheng Liu, Peitian Zhang, and Niklas Muennighoff. C-Pack: Packaged resources to advance general Chinese embedding. *arXiv preprint arXiv:2309.07597*, 2023.

A Second encoder

Collection	Condition	nDCG@10	R@100	kw empty	at ceiling	both-leg docs
SciFact	dense	0.703	0.928	–	–	–
SciFact	hybrid-and	0.705	0.928	98.0%	98.0%	0.02
SciFact	hybrid-or	0.708	0.958	0.0%	0.0%	25.9
NFCorpus	dense	0.347	0.298	–	–	–
NFCorpus	hybrid-and	0.361	0.303	50.8%	51.7%	3.7
NFCorpus	hybrid-or	0.344	0.309	7.7%	8.4%	21.0

Table 3: `nomic-embed-text-v1.5`, same collections and layout as Table 2. The BM25-only rows are encoder-independent and identical to Table 2.

B Run details

Both encoders ran on CPU (12-core x86-64, PyTorch CPU wheels) in about an hour per collection; document embeddings are cached by the runner. `nomic-embed-text-v1.5` needs `transformers` below 4.50 because its remote model code calls a method removed in later releases; the runner pins it. Documents were embedded as title and text joined by a space with the encoder's document prefix, queries with its query prefix; both normalised. The FTS5 index holds title and text columns, BM25 with default weights, depth 100 for every retriever.